

Cyfrowe Przetwarzanie Obrazu

Segmentacja obrazu

Zadanie:

Zadanie dla Państwa przedstawia się następująco z pliku sunflowers.jpg za pomocą segmentacji wyodrębnić niebo oraz kwiaty (bez liści) dwoma przedstawionymi metodami.

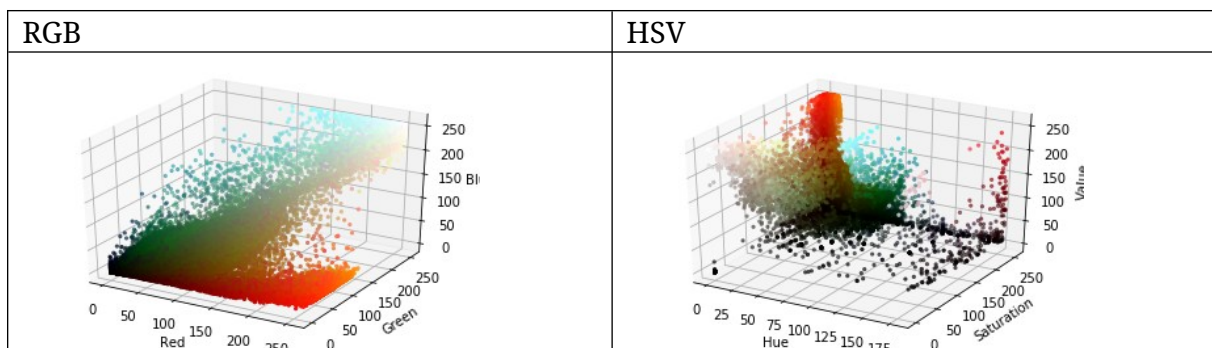
Metoda segmentacji kolorów HSV

Segmentacja obrazu pozwala na podzieleniu obrazu na pewne obszary ze względu na kolor obiektów. Mamy przykładowo obraz



Dominującymi kolorami są pomarańczowy, biały oraz żółto-zielony. Wycinając z obrazu jedną z tych barw możemy wyodrębnić jeden z głównych elementów na obrazie i dalej poddać go analizie. Na dzisiejszych zajęciach zajmiemy się sposobem jak możemy taki poddać segmentacji dzieląc go wg kolorów.

Przy operacji segmentacji w openCV wykorzystywany jest model barw HSV, wiąże się to z faktem łatwiejszego rozróżnienia barw w tym systemie kolorów.



Każda z barw tworzy chmurę punktów dla której możemy zbudować przedziały, które posłużą jako maska. Następnie wykonamy operację bitową and, która odrzuci piksele

które nie znajdują się w danym przedziale. Taka operacja pozwoli na pozostawienie tylko obiektu lub obiektów o konkretnych barwach.

Zacniemy od wykresów 3d co by mieć ogłąd obrazu:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import colors

img = cv2.imread("nemo.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

r, g, b = cv2.split(img)

fig = plt.figure()
axis = fig.add_subplot(1, 1, 1, projection="3d")
pixel_colors = img.reshape((np.shape(img)[0] * np.shape(img)[1], 3))
norm = colors.Normalize(vmin=-1.0, vmax=1.0)
norm.autoscale(pixel_colors)
pixel_colors = norm(pixel_colors).tolist()

axis.scatter( r.flatten(), g.flatten(), b.flatten(), facecolors=pixel_colors, marker=".")
axis.set_xlabel("Red")
axis.set_ylabel("Green")
axis.set_zlabel("Blue")
plt.show()

hsv_img = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)
h, s, v = cv2.split(hsv_img)

fig = plt.figure()
axis = fig.add_subplot(1, 1, 1, projection="3d")

axis.scatter(
    h.flatten(), s.flatten(), v.flatten(), facecolors=pixel_colors, marker="."
)
axis.set_xlabel("Hue")
axis.set_ylabel("Saturation")
axis.set_zlabel("Value")
plt.show()
```

Powyższy kod zamienia obraz wczytany z RGB na HSV a następnie wizualizuje poszczególne barwy w formie wykresu 3d.

Mając te informacje możemy wyznaczyć granice poszczególnych barw na początek pomarańczowy. Dla danego koloru wyznaczamy dolną i górną granicę danego koloru w HSV. Uwaga to nie jest tradycyjny HSV mamy inne granice proszę przyjrzeć się wykresowi HSV. (kod źródłowy dopisujemy poniżej pierwszego kodu).

```
light_orange = (1, 190, 200)
dark_orange = (18, 255, 255)
```

Kolejną częścią jest stworzenie maski

```
mask_orange = cv2.inRange(hsv_img, light_orange, dark_orange)
```

oraz wykonanie operacji bitowej and

```
result_orange = cv2.bitwise_and(img, img, mask=mask_orange)
```

Pozostała prezentacja wyniku

```
plt.subplot(1, 2, 1)
plt.imshow(mask_orange, cmap="gray")
plt.subplot(1, 2, 2)
plt.imshow(result_orange)
plt.show()
```

W zmiennej mask_orange mamy bitowo oznaczony obiekt na białą, natomiast w result_orange mamy wynik operacji bitowej maski z obrazem.

Jeżeli nasz obiekt składa się z dwóch kolorów jak nasza rybka możemy wykonać drugą segmentację dla koloru białego i połączyć obie w jeden wynik.

```
light_white = (0, 0, 200)
dark_white = (145, 60, 255)

mask_white = cv2.inRange(hsv_img, light_white, dark_white)
result_white = cv2.bitwise_and(img, img, mask=mask_white)
plt.subplot(1, 2, 1)
plt.imshow(mask_white, cmap="gray")
plt.subplot(1, 2, 2)
plt.imshow(result_white)
plt.show()

final_mask = mask_orange + mask_white #połączenie obu masek w jedną
```

```
final_result = cv2.bitwise_and(img, img, mask=final_mask)
plt.subplot(1, 2, 1)
plt.imshow(final_mask, cmap="gray")
plt.subplot(1, 2, 2)
plt.imshow(final_result)
plt.show()
```

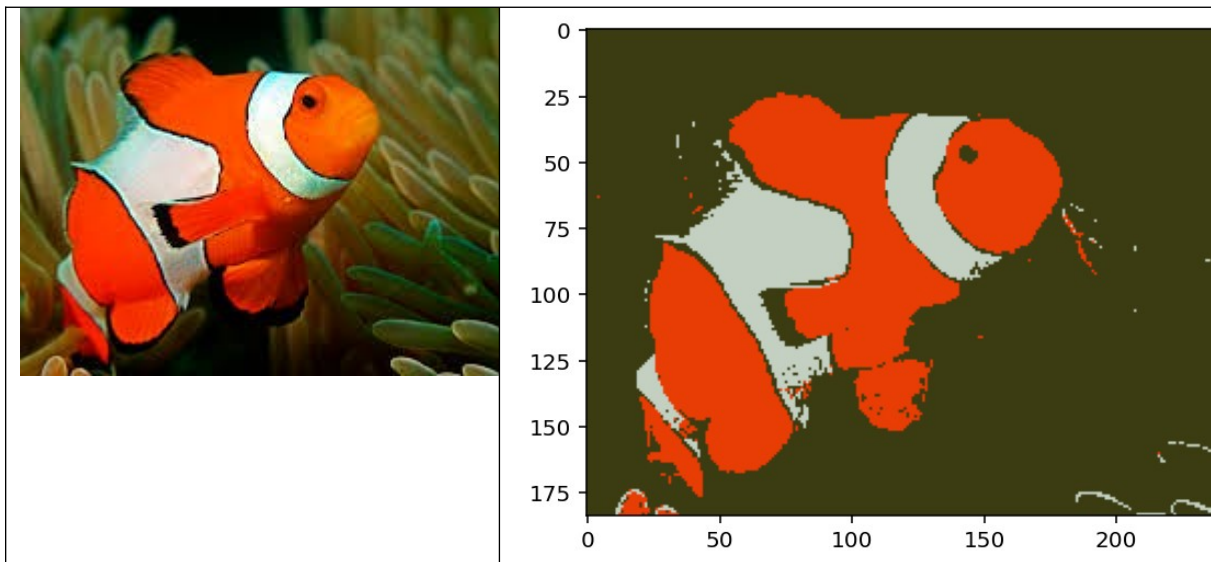
Źródło: <https://realpython.com/python-opencv-color-spaces/>

Metoda K-średnich (Algorytm centroidów)

Metoda dzieląca wszystkie punkty kolorów na wyznaczoną liczbę przedziałów. Szczegóły metody opisano poniżej https://en.wikipedia.org/wiki/K-means_clustering.

Dobrze działa na obrazach typu clip-art oraz kiedy mamy małą lu=liczbę kolorów na obrazie (o różnych odcieniach). W wyniku otrzymujemy obraz maskę przedstawiającą wynik segmentacji.

Dokładność metody określa się poprzez liczbę prób, jakie ma wykonać algorytm w celu wyznaczenia przedziałów. Im więcej prób tym dokładniejszy wynik zwykle 10 prób wystarcza.



Kod programu

```
import numpy as np
import cv2 as cv
img = cv.imread('home.jpg')
Z = img.reshape((-1,3))
# convert to np.float32
Z = np.float32(Z)
# define criteria, number of clusters(K) and apply kmeans()
criteria = (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 10, 1.0)
K = 8
```

```
ret,label,center=cv.kmeans(Z,K,None,criteria,10,cv.KMEANS_RANDOM_CENTERS)
# Now convert back into uint8, and make original image
center = np.uint8(center)
res = center[label.flatten()]
res2 = res.reshape((img.shape))
cv.imshow('res2',res2)
cv.waitKey(0)
cv.destroyAllWindows()
```

Źródło: https://docs.opencv.org/4.x/d1/d5c/tutorial_py_kmeans_opencv.html